

database\db_manager.py

```
import os
import psycopg2
from psycopg2.extras import RealDictCursor, execute_batch
from psycopg2 import pool
from typing import List, Dict, Any, Optional
import json
from datetime import datetime
import config

class DatabaseManager:
    """Manages PostgreSQL database connections with connection pooling - MEMORY OPTIMIZED"""
    def __init__(self):
        # Get database URL from environment variable
        self.database_url = os.environ.get('DATABASE_URL')
        self.connection_pool = None
        self.use_database = bool(self.database_url)
        self.cached_database_tables = {} # Track what tables exist to avoid redundant checks

        if self.use_database:
            print("[Database] Database URL found, database mode ENABLED")
            self.init_connection_pool()
            self.init_tables()
        else:
            print("[Database] No DATABASE_URL found, database mode DISABLED (using memory)")

    def init_connection_pool(self):
        """Initialize connection pool with MINIMAL connections"""
        if not self.database_url:
            return False

        try:
            # Fix URL format
            database_url = self.database_url
            if database_url.startswith('postgres://'):
                database_url = database_url.replace('postgres://', 'postgresql://', 1)

            # Create connection pool with MINIMAL connections to save memory
            self.connection_pool = pool.SimpleConnectionPool(
                1, # Min connections
                2, # Max connections (REDUCED from 3 to 2)
                database_url
            )
            print("[Database] Connection pool initialized (1-2 connections)")
            return True
        except Exception as e:
            print(f"[Database] Connection pool error: {e}")
            self.use_database = False
            return False

    def get_connection(self):
        """Get connection from pool"""
        if self.connection_pool:
            try:
                return self.connection_pool.getconn()
            except Exception as e:
                print(f"[Database] Error getting connection: {e}")
                return None

    def return_connection(self, conn):
        """Return connection to pool"""
        if self.connection_pool and conn:
            try:
                self.connection_pool.putconn(conn)
            except Exception as e:
                print(f"[Database] Error returning connection: {e}")

    def init_tables(self):
        """Initialize database tables if they don't exist"""
        conn = self.get_connection()
        if not conn:
            return
```

```

try:
    cursor = conn.cursor()

    # CVE table
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS cves (
    id VARCHAR(50) PRIMARY KEY,
    description TEXT,
    severity VARCHAR(20),
    cvss_score FLOAT,
    cwe VARCHAR(50),
    published VARCHAR(50),
    last_modified VARCHAR(50),
    reference_links JSONB,
    products JSONB,
    metrics JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
    """)
    self.cached_database_tables['cves'] = True

    # Create indexes
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_cves_severity ON cves(severity)")
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_cves_published ON cves(published)")
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_cves_cwe ON cves(cwe)")

    # Cache metadata table
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS cache_metadata (
    key VARCHAR(100) PRIMARY KEY,
    last_updated TIMESTAMP,
    total_records INTEGER,
    metadata JSONB
    )
    """)
    self.cached_database_tables['cache_metadata'] = True

    # Timeline data - NEW table for storing timeline data
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS timeline_data (
    year INTEGER,
    month INTEGER,
    count INTEGER,
    PRIMARY KEY (year, month)
    )
    """)
    self.cached_database_tables['timeline_data'] = True

    # Summary stats table - NEW table for storing pre-calculated stats
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS summary_stats (
    key VARCHAR(100) PRIMARY KEY,
    count INTEGER,
    last_updated TIMESTAMP,
    data JSONB
    )
    """)
    self.cached_database_tables['summary_stats'] = True

    conn.commit()
    print("[Database] Tables initialized successfully")
except Exception as e:
    print(f"[Database] Error initializing tables: {e}")
    conn.rollback()
finally:
    cursor.close()
    self.return_connection(conn)

def save_cves_batch(self, cves: List[Dict[str, Any]]) -> bool:
    """Save multiple CVEs to database - MEMORY OPTIMIZED"""
    if not self.use_database or not cves:
        return False

    conn = self.get_connection()
    if not conn:
        return False

```

```

try:
    cursor = conn.cursor()

    # Prepare data for batch insert - LIMIT reference_links to save space
    values = []
    for cve in cves:
        # Limit references to first 3 to save database space
        refs = cve.get('References', [])[:3]

        values.append((
            cve.get('ID', ''),
            cve.get('Description', ''),
            cve.get('Severity', 'UNKNOWN'),
            cve.get('CVSS_Score'),
            cve.get('CWE'),
            cve.get('Published', ''),
            cve.get('lastModified', ''),
            json.dumps(refs),
            json.dumps([]), # Empty products to save space
            json.dumps({}) # Empty metrics to save space
        ))

    # Use ON CONFLICT to update existing records
    batch_size = min(100, len(values)) # Process in smaller batches to manage memory
    for i in range(0, len(values), batch_size):
        batch_values = values[i:i+batch_size]
        execute_batch(cursor, """
INSERT INTO cves (id, description, severity, cvss_score, cwe,
published, last_modified, reference_links, products, metrics)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
ON CONFLICT (id) DO UPDATE SET
description = EXCLUDED.description,
severity = EXCLUDED.severity,
cvss_score = EXCLUDED.cvss_score,
cwe = EXCLUDED.cwe,
published = EXCLUDED.published,
last_modified = EXCLUDED.last_modified,
reference_links = EXCLUDED.reference_links,
products = EXCLUDED.products,
metrics = EXCLUDED.metrics,
updated_at = CURRENT_TIMESTAMP
""", batch_values)

    conn.commit()
    print(f"[Database] Saved {len(cves)} CVEs to database")
    return True
except Exception as e:
    print(f"[Database] Error saving CVEs: {e}")
    conn.rollback()
    return False
finally:
    cursor.close()
    self.return_connection(conn)

def get_cves_by_filter(self, year=None, month=None, day=None, severity=None, limit=1000)
-> List[Dict[str, Any]]:
    """Get CVEs with filters - NEW optimized method"""
    if not self.use_database:
        return []

    conn = self.get_connection()
    if not conn:
        return []

    try:
        cursor = conn.cursor(cursor_factory=RealDictCursor)

        # Build query based on filters
        query = """
SELECT id, description, severity, cvss_score, cwe,
published, last_modified, reference_links, products, metrics
FROM cves
WHERE 1=1
"""
        params = []

```

```

if year:
    query += " AND published LIKE %s"
    params.append(f"{year}%")

if month:
    query += " AND published LIKE %s"
    month_str = f"{int(month):02d}"
    params.append(f"{year}-{month_str}%")

if day:
    query += " AND published LIKE %s"
    day_str = f"{int(day):02d}"
    params.append(f"{year}-{month_str}-{day_str}%")

if severity:
    query += " AND severity = %s"
    params.append(severity.upper())

query += " ORDER BY published DESC LIMIT %s"
params.append(limit)

cursor.execute(query, params)
rows = cursor.fetchall()

# Convert to list of dicts
cves = []
for row in rows:
    cve = {
        'ID': row['id'],
        'Description': row['description'],
        'Severity': row['severity'],
        'CVSS_Score': row['cvss_score'],
        'CWE': row['cwe'],
        'Published': row['published'],
        'lastModified': row['last_modified'],
        'References': row['reference_links'] if isinstance(row['reference_links'], list) else [],
        'Products': row['products'] if isinstance(row['products'], list) else [],
        'metrics': row['metrics'] if isinstance(row['metrics'], dict) else {}
    }
    cves.append(cve)

return cves
except Exception as e:
    print(f"[Database] Error fetching CVEs with filter: {e}")
    return []
finally:
    cursor.close()
    self.return_connection(conn)

def get_all_cves(self, limit=5000) -> List[Dict[str, Any]]:
    """Get all CVEs from database - LIMIT added to prevent memory issues"""
    if not self.use_database:
        return []

    conn = self.get_connection()
    if not conn:
        return []

    try:
        cursor = conn.cursor(cursor_factory=RealDictCursor)
        cursor.execute("""
        SELECT id, description, severity, cvss_score, cwe,
        published, last_modified, reference_links, products, metrics
        FROM cves
        ORDER BY published DESC
        LIMIT %s
        """, (limit,))

    rows = cursor.fetchall()

    # Convert to list of dicts
    cves = []
    for row in rows:
        cve = {
            'ID': row['id'],
            'Description': row['description'],

```

```

'Severity': row['severity'],
'CVSS_Score': row['cvss_score'],
'CWE': row['cwe'],
'Published': row['published'],
'lastModified': row['last_modified'],
'References': row['reference_links'] if isinstance(row['reference_links'], list) else
[],
'Products': row['products'] if isinstance(row['products'], list) else [],
'metrics': row['metrics'] if isinstance(row['metrics'], dict) else {}
}
cves.append(cve)

return cves
except Exception as e:
print(f"[Database] Error fetching CVEs: {e}")
return []
finally:
cursor.close()
self.return_connection(conn)

def get_recent_cves(self, days=30, limit=5000) -> List[Dict[str, Any]]:
"""Get CVEs from the last N days - NEW optimized method"""
if not self.use_database:
return []

conn = self.get_connection()
if not conn:
return []

try:
cursor = conn.cursor(cursor_factory=RealDictCursor)

# Use a simplified query that doesn't require complex date operations
cursor.execute("""
SELECT id, description, severity, cvss_score, cwe,
published, last_modified, reference_links, products, metrics
FROM cves
WHERE published > (CURRENT_DATE - INTERVAL '%s days')::TEXT
ORDER BY published DESC
LIMIT %s
""", (days, limit))

rows = cursor.fetchall()

# Convert to list of dicts
cves = []
for row in rows:
cve = {
'ID': row['id'],
'Description': row['description'],
'Severity': row['severity'],
'CVSS_Score': row['cvss_score'],
'CWE': row['cwe'],
'Published': row['published'],
'lastModified': row['last_modified'],
'References': row['reference_links'] if isinstance(row['reference_links'], list) else
[],
'Products': row['products'] if isinstance(row['products'], list) else [],
'metrics': row['metrics'] if isinstance(row['metrics'], dict) else {}
}
cves.append(cve)

return cves
except Exception as e:
print(f"[Database] Error fetching recent CVEs: {e}")
return []
finally:
cursor.close()
self.return_connection(conn)

def update_cache_metadata(self, key: str, total_records: int, metadata: Dict = None):
"""Update cache metadata"""
if not self.use_database:
return

conn = self.get_connection()
if not conn:

```

```

return

try:
    cursor = conn.cursor()
    cursor.execute("""
INSERT INTO cache_metadata (key, last_updated, total_records, metadata)
VALUES (%s, CURRENT_TIMESTAMP, %s, %s)
ON CONFLICT (key) DO UPDATE SET
last_updated = CURRENT_TIMESTAMP,
total_records = EXCLUDED.total_records,
metadata = EXCLUDED.metadata
""", (key, total_records, json.dumps(metadata or {})))
    conn.commit()
except Exception as e:
    print(f"[Database] Error updating metadata: {e}")
    conn.rollback()
finally:
    cursor.close()
self.return_connection(conn)

def get_cache_metadata(self, key: str) -> Optional[Dict]:
    """Get cache metadata"""
    if not self.use_database:
        return None

    conn = self.get_connection()
    if not conn:
        return None

    try:
        cursor = conn.cursor(cursor_factory=RealDictCursor)
        cursor.execute("""
SELECT last_updated, total_records, metadata
FROM cache_metadata
WHERE key = %s
""", (key,))
        row = cursor.fetchone()

        if row:
            return {
                'last_updated': row['last_updated'],
                'total_records': row['total_records'],
                'metadata': row['metadata']
            }

        return None
    except Exception as e:
        print(f"[Database] Error fetching metadata: {e}")
        return None
    finally:
        cursor.close()
    self.return_connection(conn)

def save_timeline_data(self, year_month_counts):
    """Save timeline data to database - NEW"""
    if not self.use_database:
        return False

    conn = self.get_connection()
    if not conn:
        return False

    try:
        cursor = conn.cursor()
        values = []

        for (year, month), count in year_month_counts.items():
            values.append((year, month, count))

        execute_batch(cursor, """
INSERT INTO timeline_data (year, month, count)
VALUES (%s, %s, %s)
ON CONFLICT (year, month) DO UPDATE SET
count = EXCLUDED.count
""", values)

        conn.commit()

```

```

print(f"[Database] Saved timeline data: {len(values)} records")
return True
except Exception as e:
print(f"[Database] Error saving timeline data: {e}")
conn.rollback()
return False
finally:
cursor.close()
self.return_connection(conn)

def get_timeline_data(self, years=1) -> Dict[str, Any]:
"""Get timeline data from database - NEW"""
if not self.use_database:
return {'labels': [], 'values': [], 'total_cves': 0, 'months_covered': 0, 'raw_data':
{}}

conn = self.get_connection()
if not conn:
return {'labels': [], 'values': [], 'total_cves': 0, 'months_covered': 0, 'raw_data':
{}}

try:
cursor = conn.cursor()

# Get current year
current_year = datetime.now().year

# Calculate start year
start_year = current_year - years

cursor.execute("""
SELECT year, month, count
FROM timeline_data
WHERE year >= %s
ORDER BY year, month
""", (start_year,))

rows = cursor.fetchall()

# Process into timeline format
labels = []
values = []
raw_data = {}
total_cves = 0

for year, month, count in rows:
label = f"{year}-{month:02d}"
labels.append(label)
values.append(count)
raw_data[label] = count
total_cves += count

return {
'labels': labels,
'values': values,
'total_cves': total_cves,
'months_covered': len(labels),
'raw_data': raw_data
}
except Exception as e:
print(f"[Database] Error fetching timeline data: {e}")
return {'labels': [], 'values': [], 'total_cves': 0, 'months_covered': 0, 'raw_data':
{}}
finally:
cursor.close()
self.return_connection(conn)

def save_summary_stats(self, key, count, data):
"""Save summary statistics to database - NEW"""
if not self.use_database:
return False

conn = self.get_connection()
if not conn:
return False

try:

```

```

cursor = conn.cursor()
cursor.execute("""
INSERT INTO summary_stats (key, count, last_updated, data)
VALUES (%s, %s, CURRENT_TIMESTAMP, %s)
ON CONFLICT (key) DO UPDATE SET
count = EXCLUDED.count,
last_updated = CURRENT_TIMESTAMP,
data = EXCLUDED.data
""", (key, count, json.dumps(data)))

conn.commit()
return True
except Exception as e:
print(f"[Database] Error saving summary stats: {e}")
conn.rollback()
return False
finally:
cursor.close()
self.return_connection(conn)

def get_summary_stats(self, key):
"""Get summary statistics from database - NEW"""
if not self.use_database:
return None

conn = self.get_connection()
if not conn:
return None

try:
cursor = conn.cursor(cursor_factory=RealDictCursor)
cursor.execute("""
SELECT count, last_updated, data
FROM summary_stats
WHERE key = %s
""", (key,))

row = cursor.fetchone()
if row:
return {
'count': row['count'],
'last_updated': row['last_updated'],
'data': row['data']
}
return None
except Exception as e:
print(f"[Database] Error getting summary stats: {e}")
return None
finally:
cursor.close()
self.return_connection(conn)

def clear_all_cves(self):
"""Clear all CVE data (for refresh)"""
if not self.use_database:
return

conn = self.get_connection()
if not conn:
return

try:
cursor = conn.cursor()
cursor.execute("DELETE FROM cves")
conn.commit()
print("[Database] Cleared all CVEs from database")
except Exception as e:
print(f"[Database] Error clearing CVEs: {e}")
conn.rollback()
finally:
cursor.close()
self.return_connection(conn)

def get_stats(self) -> Dict[str, Any]:
"""Get database statistics"""
if not self.use_database:
return {}

```

```

conn = self.get_connection()
if not conn:
    return {}

try:
    cursor = conn.cursor()
    cursor.execute("SELECT COUNT(*) FROM cves")
    total_cves = cursor.fetchone()[0]

    # Get table sizes
    cursor.execute("""
SELECT relname as table, pg_size_pretty(pg_total_relation_size(relid)) as size
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC
""")
    tables = {row[0]: row[1] for row in cursor.fetchall()}

    return {
        'total_cves': total_cves,
        'database_enabled': True,
        'table_sizes': tables
    }
except Exception as e:
    print(f"[Database] Error getting stats: {e}")
    return {}
finally:
    cursor.close()
    self.return_connection(conn)

def close(self):
    """Close all connections in pool"""
    if self.connection_pool:
        self.connection_pool.closeall()
    print("[Database] Connection pool closed")

# Global database manager instance
db_manager = DatabaseManager()

```